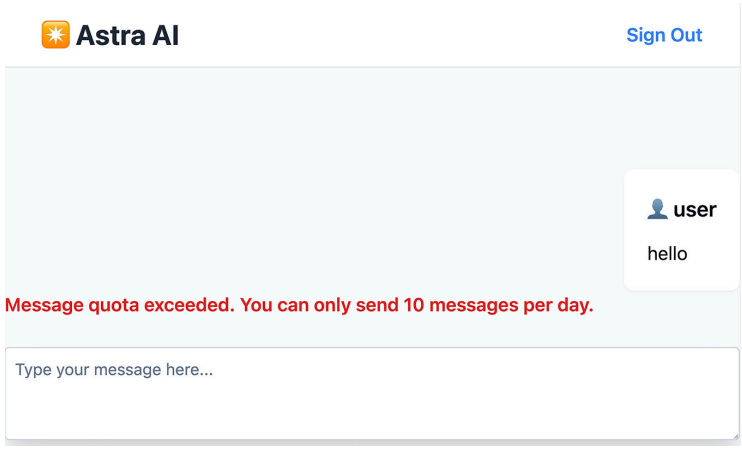


Figure 9.4 Overview of the message quota implementation. The top part displays the Upstash Data Browser, showing the Redis key and value tracking the number of messages sent by a user on a given day. The bottom part shows the API response, indicating a 429 Too Many Requests status code when the user exceeds their daily message limit of 10 messages.

> **WARNING** While implementing message limiting helps manage user interactions, determined individuals may exploit disposable email services to create multiple accounts and bypass these limits. To enhance the security of registered users, implement invitation-only registration to restrict access and reduce the risk of abuse from disposable emails. Utilize unique invitation codes, require email verification, and monitor registration patterns to detect suspicious activity. Consider requiring business email addresses for registration and using CAPTCHA to deter automated submissions. This will help you avoid excessive use of API credits.

Now that we have some good practices in place for managing message quotas and Redis keys, we turn our attention to another critical aspect of application security: API key and secrets management.

9.4 API key and secrets management

Managing API keys and secrets securely is very important for any Next.js application, especially when working with services like OpenAI. Having poor secrets management can lead to costly security breaches, leaked credentials, and compromised user data. Clearly, we need to be familiar with best practices and common pitfalls in handling sensitive credentials.

9.4.1 Understanding Next.js environment variables

So far, we've been placing our secrets and environment variables into an .env file. However, Next.js provides two different ways to define them, with each serving different purposes: